

Souvenirs

Amaru está comprando recuerdos en una tienda en el extranjero. Hay N **tipos** de souvenirs. En la tienda hay infinitos souvenirs de cada tipo disponible.

Cada tipo de souvenir tiene un precio fijo. En concreto, un souvenir de tipo i ($0 \leq i < N$) tiene un precio de $P[i]$ monedas, donde $P[i]$ es un entero positivo.

Amaru sabe que los tipos de souvenirs están ordenados por precio en orden decreciente, y que no hay dos precios iguales. Concretamente, $P[0] > P[1] > \dots > P[N-1] > 0$. Además, pudo conocer el valor de $P[0]$. Desafortunadamente Amaru no tiene más información sobre los precios de los souvenirs.

Para comprar algunos souvenirs, Amaru realizará una serie de transacciones con el vendedor.

Cada transacción consta de los siguientes pasos:

1. Amaru entrega una cantidad (positiva) de monedas al vendedor.
2. El vendedor pone estas monedas en una pila sobre la mesa en la trastienda, donde Amaru no puede verlas.
3. El vendedor tiene en consideración cada souvenir de tipo $0, 1, \dots, N-1$, en ese orden, uno por uno. Cada tipo de souvenir se tiene en consideración **exactamente una vez** en cada transacción.
 - Cuando tiene en consideración el tipo de souvenir i , si el cantidad actual de monedas en la pila es al menos $P[i]$, entonces
 - el vendedor retira $P[i]$ monedas de la pila, y
 - el vendedor pone un souvenir de tipo i sobre la mesa.
4. El vendedor entrega a Amaru todas las monedas que quedan en la pila y todos los souvenirs que están sobre la mesa.

Ten en cuenta que no hay monedas ni souvenirs en la mesa antes de comenzar las transacciones.

Tu misión es dar instrucciones a Amaru para que realice una serie de transacciones de manera que:

- en cada transacción compra **al menos un** souvenir, y
- en total compra **exactamente** i souvenirs de tipo i , para cada i tal que $0 \leq i < N$. Ten en cuenta que esto significa que Amaru no debe comprar ningún souvenir de tipo 0 .

Amaru no tiene por qué minimizar el número de transacciones, y tiene monedas infinitas.

Detalles de implementación

Debes implementar la siguiente función.

```
void buy_souvenirs(int N, long long P0)
```

- N : el número de tipos de souvenirs.
- $P0$: el valor de $P[0]$.
- Esta función se llama exactamente una vez para cada caso de prueba.

La función anterior puede realizar llamadas a la siguiente función para dar instrucciones a Amaru para realizar una transacción:

```
std::pair<std::vector<int>, long long> transaction(long long M)
```

- M : la cantidad de monedas entregadas por Amaru al vendedor.
- La función devuelve un `pair`. El primer elemento del pair es un vector L que contiene los tipos de souvenirs que se han comprado (en orden creciente). El segundo elemento es un entero R , la cantidad de monedas devueltas a Amaru después de la transacción.
- Se requiere que $P[0] > M \geq P[N - 1]$. La condición $P[0] > M$ asegura que Amaru no compre ningún souvenir de tipo 0, y $M \geq P[N - 1]$ asegura que Amaru compre al menos un souvenir. Si no se cumplen estas condiciones, tu solución recibirá el veredicto `Output isn't correct: Invalid argument`. Fíjate que, a diferencia de $P[0]$, el valor de $P[N - 1]$ no se proporciona en la entrada.
- La función se puede llamar como máximo 5000 veces para cada caso de prueba.

El comportamiento del calificador **no es adaptativo**. Esto significa que la secuencia de precios P es fija antes de llamar a `buy_souvenirs`.

Restricciones

- $2 \leq N \leq 100$
- $1 \leq P[i] \leq 10^{15}$ para cada i tal que $0 \leq i < N$.
- $P[i] > P[i + 1]$ para cada i tal que $0 \leq i < N - 1$.

Subtareas

Subtarea	Puntuación	Restricciones adicionales
1	4	$N = 2$
2	3	$P[i] = N - i$ para cada i tal que $0 \leq i < N$.
3	14	$P[i] \leq P[i + 1] + 2$ para cada i tal que $0 \leq i < N - 1$.
4	18	$N = 3$
5	28	$P[i + 1] + P[i + 2] \leq P[i]$ para cada i tal que $0 \leq i < N - 2$. $P[i] \leq 2 \cdot P[i + 1]$ para cada i tal que $0 \leq i < N - 1$.
6	33	Sin restricciones adicionales.

Ejemplo

Considera la siguiente llamada:

```
buy_souvenirs(3, 4)
```

Hay $N = 3$ tipos de souvenirs y $P[0] = 4$. Observa que sólo hay tres posibles secuencias de precios P : $[4, 3, 2]$, $[4, 3, 1]$, y $[4, 2, 1]$.

Supongamos que `buy_souvenirs` llama a `transaction(2)`. Supongamos que la llamada devuelve $([2], 1)$, lo que significa que Amaru compró un souvenir de tipo 2 y el vendedor le devolvió 1 moneda. Observa que esto nos permite deducir que $P = [4, 3, 1]$, ya que:

- Para $P = [4, 3, 2]$, `transaction(2)` habría devuelto $([2], 0)$.
- Para $P = [4, 2, 1]$, `transaction(2)` habría devuelto $([1], 0)$.

Entonces `buy_souvenirs` puede llamar a `transaction(3)`, que devuelve $([1], 0)$, lo que significa que Amaru compró un souvenir de tipo 1 y el vendedor le devolvió 0 monedas. Hasta ahora, en total, ha comprado un souvenir de tipo 1 y un souvenir del tipo 2.

Finalmente, `buy_souvenirs` puede llamar a `transaction(1)`, que devuelve $([2], 0)$, lo que significa que Amaru compró un souvenir de tipo 2. Ten en cuenta que también podríamos haber utilizado `transaction(2)` aquí. En este punto, en total Amaru tiene un souvenir de tipo 1 y dos souvenirs del tipo 2, como requiere el ejercicio.

Sample Grader

Formato de entrada:

N

$P[0] \ P[1] \ \dots \ P[N-1]$

Formato de salida:

$Q[0] \ Q[1] \ \dots \ Q[N-1]$

Aquí $Q[i]$ es el número de souvenirs de tipo i comprados en total para cada i tal que $0 \leq i < N$.